

RESEARCH ARTICLE

Bilateral-Weighted Online Adaptive Isolation Forest for anomaly detection in streaming data

Gábor Hannák¹ | Gábor Horváth² | Attila Kádár² | Márk Dániel Szalai²

¹Nokia Bell Labs, Budapest, Hungary

²Department of Networked Systems and Services, Budapest University of Technology and Economics, Budapest, Hungary

Correspondence

Gábor Horváth, Department of Networked Systems and Services, Budapest University of Technology and Economics, Műegyetem rkp. 3, Budapest, H-1111 Hungary.
Email: horvath.gabor@vik.bme.hu

Funding information

Artificial Intelligence National Laboratory, Hungary; OTKA 138208, Hungary

Abstract

We propose a method called Bilateral-Weighted Online Adaptive Isolation Forest (BWOAIF) for unsupervised anomaly detection based on *Isolation Forest* (IF), which is applicable to streaming data and able to cope with concept drift. Similar to IF, the proposed method has only few hyperparameters whose effect on the performance are easy to interpret by human intuition and therefore easy to tune. BWOAIF ingests data and classifies it as normal or anomalous, and simultaneously adapts its classifier by removing old trees as well as by creating new ones. We show that BWOAIF adapts gradually to slow concept drifts, and, at the same time, it is able to adapt fast to sudden changes of the data distribution. Numerical results show the efficacy of the proposed algorithm and its ability to learn different classes of concept drifts, such as slow/fast concept shift, concept split, concept appearance, and concept disappearance.

KEYWORDS

anomaly detection, bilateral weighting, ensemble methods, isolation forest, online machine learning, outliers, stream processing

1 | INTRODUCTION

Anomaly detection refers to the method or process of finding irregular elements or patterns in a data set. These can be outliers, extremes, or elements that otherwise deviate from the normal behavior and are therefore of high interest to the observer. Application areas of anomaly detection reach from medical imaging (e.g., tumor detection in MRI) [8, 12] through computer networks (e.g., intrusion or fault detection) [5] all the way to sensor signal processing (e.g., fault detection) [3], and it can provide insight in many scenarios where massive amounts of data are available. There is no canonical definition of anomaly but its definition changes from context to context, based on the

requirements and the application setting. But even within a certain context it can change as new data are generated by the underlying process, or because the system model may vary. Many anomaly detection methods are specific to a very narrow context, while others are more general and rely less on assumptions on the data.

Anomaly detection has received substantial attention from the machine learning community, as the amount, the variety, and the pace of the generated data prohibit methods involving major human intervention. As in machine learning, anomaly detection methods can be categorized into supervised, unsupervised, and semi-supervised methods, depending on the availability of labels in the training data set. There are several approaches to defining

and detecting anomalies, such as classification/clustering based, nearest neighbor based, statistical, information theory based, and spectral. For comprehensive surveys on related work the interested reader is referred to [1, 2, 6].

In offline anomaly detection, the data in which anomalies are to be detected are fully available during the design of the anomaly detection method and therefore the method can be comprehensively tailored to the data. In contrast, in many real-world scenarios data arrives sequentially, with individual data elements or data batches to be processed immediately, or in regular or irregular time intervals. This is referred to as *online* or *streaming* anomaly detection. Given that the characteristics of the new data does not change with time, any anomaly detection method will work in an online scenario once trained on an initial, sufficiently large batch of data and then executed on the new data. However, if anomaly detection has to account for changes in the input data characteristics, that is, concept drift, it has to be able to adapt dynamically during inference.

In this paper, we propose an extension of the Isolation Forest (IF) algorithm [9], Bilateral-Weighted Online Adaptive Isolation Forest (BWOAIF), which is able to cope with various kinds of concept drifts and adapt its classifier accordingly during inference. Similar to the original IF algorithm, BWOAIF has low computational requirements and is thus scalable, has low number of hyperparameters, and has wide applicability due to its rather general anomaly definition.

The rest of this paper is organized as follows. Section 2 discusses two topics which are important in this paper, namely isolation forests and the machine learning methods available to cope with concept drift. The proposed method is introduced and described in Section 3. Some advice on the proper setting of the model parameters are provided in Section 4. In Section 5, we present numerical experiments using synthetic and real data sets to demonstrate the strengths of the new algorithm. Finally, Section 6 concludes the paper.

2 | RELATED WORK

2.1 | Isolation Forest

IF is a computationally efficient, decision-tree-based classifier for (multi-dimensional) numerical data. It does not rely on distance calculation and its computational and memory requirements scale linearly with the amount of data. Its anomaly detection concept relies on the observation that anomalies are more susceptible to isolation because they are distant and rare, and therefore its key procedure is isolating data instances. In the following, we give

a conceptual description of the IF method whose purpose is to provide the basis for the next section. For full details please refer to the original work [9].

Given are the data set $\mathcal{X} = \{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(N)}\}$ of size N with D -dimensional elements $\mathbf{x}^{(i)} = (x_1^{(i)}, \dots, x_D^{(i)})^T \in \mathbb{R}^D$, a subsampling size ψ , the number T of *i*Trees in the iForest, and a maximum tree depth $d_{\max}$.

2.1.1 | Training

The training phase consist of constructing T *i*Trees which comprise the iForest. Each *i*Tree acts on a distinct set S of ψ randomly chosen elements of $\mathcal{X}$ by recursively partitioning it along a binary decision tree. The partitioning is each time based on a random attribute (dimension) $q \in \{1, \dots, D\}$ and a random split value p such that the test $x_q^{(i)} \leq p$ divides data points in the current set into t_l and t_r . Each node t in the *i*Tree is either an external node with no child, or an internal node with one test and exactly two daughter nodes (t_l, t_r). The recursive partitioning is stopped when (i) the tree depth has reached $d_{\max}$, or (ii) the set to partition has cardinality 1, or (iii) all elements in the set to partition are equal.

2.1.2 | Anomaly score inference

Any $\mathbf{x} \in \mathbb{R}^D$ can be applied, that is, passed down an *i*Tree, which will result in a sequence of decisions until $\mathbf{x}$ reaches a terminating node. The path length of $\mathbf{x}$ on *i*Tree number i , denoted by $h_i(\mathbf{x})$, is the number of edges it traverses till it reaches the terminating node. W.h.p. there will be unsuccessful searches because some path lengths are limited by $d_{\max}$ rather than isolation of the given point. Given T *i*Trees, the anomaly score $s(\mathbf{x}^{(i)})$ is calculated from the T individual path lengths of $\mathbf{x}$ associated with those T *i*Trees, in such a way that there is compensation for the unsuccessful searches. For details of this calculation we refer the interested reader to [Reference 9, Section 2].

2.1.3 | Discussion

IF is shown to work best in low dimensions with T in the range 100–200 and ψ in the range 128–256. Each *i*Tree represents a weak soft classifier in that it will in expectation return a large path length for points with relatively dense neighborhood (i.e., points similar to many others), and a small path length for points with a relatively sparse neighborhood (i.e., more isolated points). The anomaly score $s(\mathbf{x}^{(i)})$ is between 0 and 1. If an anomaly score is close to

1, it signals an anomaly. If an anomaly score is close to 0, it signals normality (lack of anomaly). If most of the anomaly scores in the data set are close to 0.5, then the data set shows no distinctive anomalies. Note that the anomaly score is *not* an approximate measure of the inverse relative density. Even if anomalous points are locally more dense than normal points, if they are significantly fewer in number and also distant, they will be easier to isolate. The reason is the strong subsampling upon building the *i*Trees: in a ψ -sized randomly subsampled data set one expects to have mostly normal points and only few, if any, anomalous points. Therefore, the latter will be easy to isolate (i.e., close to the root of the *i*Tree), and the *i*Trees learn rather a model for the normal points than abnormal points.

2.2 | Machine learning methods for data with concept drift

Concept drift means that the distribution of the data varies over time [11]. The change can be

- abrupt, for instance when the data collection method has changed, the system was re-configured, or in case of a (partial) fault;
- or gradual, typically when the environment changes.

Online machine learning methods coping with concept drifts have a huge literature. Ensemble-based solutions turned out to be especially successful, since they allow partial model updates when new data arrives, avoiding a computationally expensive, complete rebuilding of the model. The vast majority of the published methods have been developed for the supervised setup. The Dynamic Weighted Majority [7] is a popular method that evaluates all members of the ensemble and adjusts their weights according to their prediction accuracy. In the model update step unsuccessful members are dropped and new models are added to the ensemble if the global performance of the model is low.

Online methods for unsupervised problems are less thoroughly researched, especially for anomaly detection. Several extensions of the IF method have been proposed that aim to cope with non-stationary data arriving as a continuous data stream. The authors of [4] propose the iForestASD algorithm which follows a window-based approach: the anomalies of the next data stream window comprising W data samples are evaluated with the current model. If the anomaly rate in this window is higher than a pre-defined threshold, the model is qualified as outdated and a new IF is generated based on this window of data. While this simple adaptation of the original IF method performs surprisingly well in many scenarios, it also has some significant flaws:

- It is blind to some kinds of concept drift. For example, when the variance of the data decreases (i.e., *shrinkage*), the model will classify all data as normal and a model update is never triggered.
- Inconsistent behavior upon adaptation. That is, because the entire model is recreated when it is qualified as outdated (based on a manually set threshold), the classifier will output very different anomaly scores for similar samples. This leads to inconsistent post-processing (e.g., manual investigation of the anomaly). Therefore, in many engineering systems models producing smooth output are more preferable.
- Continuity: It treats the streaming problem as a sequence of offline learning problems and fails to capture smooth transitions. In a way, it assumes sharp transitions between concepts.

A more recent work, [15], extends iForestASD by exploring and comparing several concept drift detection methods, based on whose output the (complete) model update is triggered. The authors point out the importance of concept drift detection *before* and *after* inference on an arriving data batch. To the best of our knowledge, all ensemble-based online anomaly detection methods follow this approach, hence, they recreate the model from scratch when deemed necessary. This approach may be beneficial in case of abrupt and substantial concept drifts, but it does not provide smooth update of the anomaly scores in case of gradual concept drifts. Our aim in this paper is to present an algorithm that handles gradual and abrupt concept drifts equally well.

3 | BILATERAL-WEIGHTED ONLINE ADAPTIVE ISOLATION FOREST

To overcome the limitations identified in Section 2, we introduce a new online IF variant (called BWOAIF) that is able to update the model continuously with arbitrary frequency and granularity. Our method is also window based. Based on a new batch of samples, the ensemble of trees is updated such that some of the oldest trees are replaced by an equal number of new trees. We also introduce an *i*Tree weighing scheme to tune the speed of the adaptation of the model to the concept drift of the data stream.

3.1 | Training

BWOAIF updates the model based on non-overlapping data batches of size B . The model update comprises adding

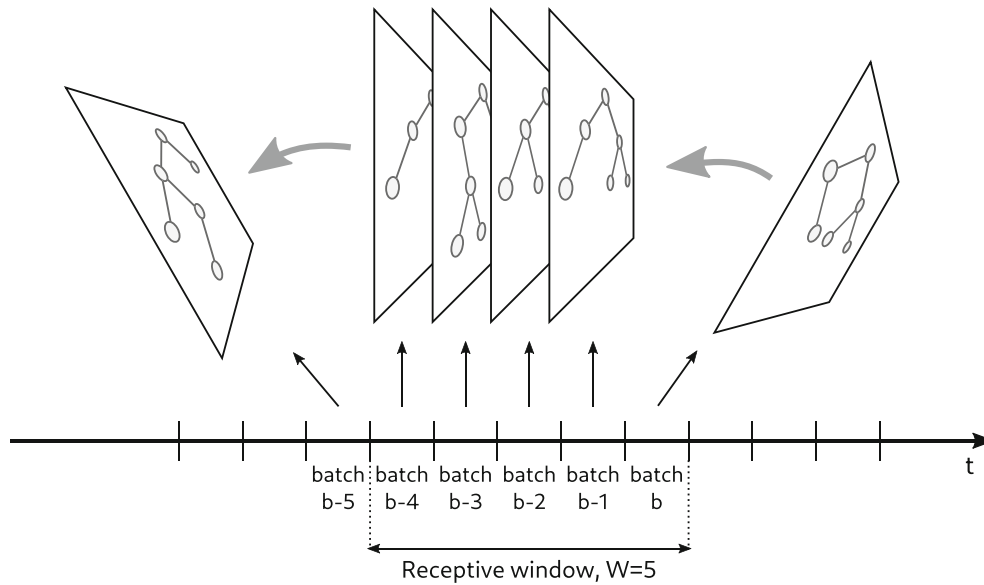


FIGURE 1 The model update of the BWOAIF algorithm with parameters $T = 5$ and $E = 1$

E new trees and removing the E oldest ones from the ensemble (ensuring the adaption to the potentially changing distribution of the data). If the total number of trees in the ensemble is T , the *receptive window* of the model (the time period covered by the model) is $W = B \cdot T/E$, since the number of batches based on which the model is built is $K = T/E$. (For simplicity, we assume that T is an integer multiple of E). The steps of the model update processing the batch ending with $\mathbf{x}^{(t)}$ ($t = W, W + B, W + 2B, \dots, N$) are as follows:

- Delete the oldest E trees from the forest, based on the timestamp of the trees.
- Add E new trees to the forest:
 - Build the trees from ψ samples, which are chosen in a non-uniform way: sample $\mathbf{x}^{(t-k \cdot B+i)}$, $i = 1, \dots, B$, $k = 1, \dots, K$ is selected with weight

$$w^{(t-k \cdot B+i)} = \gamma^{k-1},$$

hence, samples from batch k apart from the current one are selected according to geometrical weights.

- Add timestamp t to the newly created trees.

The weight parameter $\gamma \in [0, 1]$ balances between fast adaptation versus smoothness of the anomaly scores:

- With $\gamma = 0$ the new trees will be trained solely based on the samples of the latest batch, maximizing the adaptation speed of the model. Note that the effect of preceding batches is still retained by the model through the older trees in the receptive window.
- By setting $\gamma = 1$, all samples of the batches of the receptive window W are chosen with equal weight when

building the new trees. This way the anomaly scores are expected to change slower, leading to smoother output.

The concept of the training process is depicted by Figure 1.

3.2 | Anomaly score inference

In the ordinary IF method the path length in each tree contributes equally to the anomaly score when evaluating a sample $\mathbf{x}^{(i)}$. In BWOAIF we assign weights to each tree such that in case of an abrupt change in the data distribution, the outdated trees of the ensemble do not slow down the adaption. This idea is inspired by bilateral filtering, a technique commonly used in image processing for edge preserving denoising [16].

Let us partition the trees of the ensemble into groups of size E based on their age, thus group $k = 1$ consist of the E trees built from the last training batch, group $k = 2$ corresponds to the trees from the second to last training batch, etc. For group k , the anomaly score for sample $\mathbf{x}^{(t)}$ is computed as in ordinary IF, hence

$$s_k(\mathbf{x}^{(t)}) = 2^{-\frac{E_k(h(\mathbf{x}^{(t)}))}{c(\psi)}}, \quad k = 1, \dots, K,$$

where $c(\psi)$ is the average path length of trees containing ψ elements and $E_k(h(\mathbf{x}^{(t)}))$ is the average path length leading to $\mathbf{x}^{(t)}$ in the trees of the k th group. As a result, we get K anomaly scores, one for each group of trees. Next, we aggregate these results using bilateral weighting as

$$s(\mathbf{x}^{(t)}) = \frac{1}{S} \sum_{k=1}^K s_k(\mathbf{x}^{(t)}) \cdot e^{-\frac{(s_k(\mathbf{x}^{(t)}) - s_1(\mathbf{x}^{(t)}))^2}{2\sigma_s^2}} \cdot e^{-\frac{(k-1)^2}{2\sigma_a^2}},$$

where S is the normalization constant

$$S = \sum_{k=1}^K e^{-\frac{(\mathbf{y}_k(\mathbf{x}^{(t)}) - s_1(\mathbf{x}^{(t)}))^2}{2\sigma_v^2}} \cdot e^{-\frac{(k-1)^2}{2\sigma_a^2}}.$$

The rationale behind the weighting is as follows. The second exponential factor of the weight is responsible for prioritizing more recent trees over older ones. The strength of this distinction can be controlled by parameter σ_a : setting it near zero means that the contribution of the most recent trees dominates the anomaly score, while a large σ_a will equalize the contribution of all trees. The first exponential factor of the weight ensures that trees built on very different samples (probably from an outdated distribution) have less influence when computing the aggregated anomaly score. If there was an abrupt concept drift recently, mainly trees from the new distribution will be taken into account in the anomaly score. Parameter σ_v controls how much difference is tolerated between the anomaly scores.

4 | MODEL PARAMETERS

The parameters of the method are:

- B : number of samples in a batch,
- ψ : the number of random samples used for building a tree,
- E : the number of trees to be built after receiving a batch of data,
- T : the total number of trees in the BWOAIF model,
- σ_a : parameter controlling the suppression of old trees when computing anomaly scores,
- σ_v : parameter punishing dissimilar trees when computing anomaly scores.

The rule of thumb is to set ψ and E similarly to ordinary IF models. Most of the publicly available implementations use default values $\psi = 128 \dots 256$ and $E = 100 \dots 200$.

Parameter B should be set such that the distribution of the data in each batch is approximately stationary. If determining this is not possible, we recommend to set it approximately ψ .

The parameter T should be set based on the receptive window size $W = B \cdot T / E$. To provide smooth anomaly scores, a sufficiently high number of trees (T) are needed. If there are periodic concept drifts in the data, it is beneficial to set the receptive window such that at least two periods are covered, this way the old trees corresponding to the same phase of the concept drift can be reused in the computation of the anomaly scores, leading to smoother (low variance) results.

It is safe to set σ_a relatively high (not to punish old trees) in most cases. Parameter σ_v is specific to the data set, it should be set with some experimentation. Too low values should be avoided, since in this case only E trees contribute to the anomaly score, leading to oversensitivity and highly varying anomaly scores. On the other hand, too high values disable the capability of BWOAIF adapting to sudden changes of data distribution completely. Fortunately, the anomaly scores in isolation forest fall into a well-known range (around 0.5), regardless of the data set, we got excellent results with $\sigma_v \in [0.05, 0.1]$ in almost all the cases we considered.

5 | EVALUATION OF THE ALGORITHM

Evaluating unsupervised machine learning algorithms in the presence of concept drift is a challenging problem by itself. The most widely used data set for validation, *STAGGER*, consists of categorical features, which is not appropriate for benchmarking IF-based methods.

5.1 | A new synthetic data set for unsupervised anomaly detection with concept drift

To validate our method and compare it to alternative solutions, we propose a new data generation framework with $\mathbf{x}^{(i)} = (x_1^{(i)}, x_2^{(i)}) \in \mathbb{R}^2$. The proposed streaming data set generation consists of various phases (lasting for 5 s each), in which different kinds of concept drift are produced:

- Phase 1: samples $x_1^{(i)}, x_2^{(i)}$ are generated independently from a standard normal distribution.
- Phase 2, *shift*: the mean of the normal distribution along the first dimension is linearly changing from 0 to 10.
- Phase 3, *shrinkage*: the variance of the normal distribution is linearly shrinking from 1 to 0.001 along both dimensions (the covariance is zero).
- Phase 4, *expansion*: the variance of the normal distribution is linearly increasing from 0.001 to 1 along both dimensions (covariance is zero).
- Phase 5, *distribution jump*: samples $x_1^{(i)}, x_2^{(i)}$ are (suddenly) drawn from a uniform distribution such that $x_1^{(i)} \sim \text{Uni}(9, 11), x_2^{(i)} \sim \text{Uni}(-1, 1)$.
- Phase 6, *distribution jump*: the support of the uniform distribution changes suddenly such that $x_1^{(i)} \sim \text{Uni}(-2, 2), x_2^{(i)} \sim \text{Uni}(-6, -2)$.
- Phase 7, *distribution mix*: the distribution of the samples are a weighted mixture of the uniform distribution and

a normal distribution centered at $(10, -4)$ (the variance and the covariance of the latter are 1 and 0, respectively), with weight linearly increasing from 0 to 1 in the favor of the normal distribution.

- Phase 8: the samples are generated from the normal distribution with mean $(10, -4)$ (variance 1, covariance 0).

In this numerical experiment we set the batch size to $B = 64$, the number of trees per batch to $E = 64$ and the total number of trees to $T = 1024$. Thus, the receptive window consists of $W = 1024$ samples amounting to 16 batches. The number of samples was 40,000. We execute the following methods 50 times on the synthetic data set with different random seeds:

- An algorithm that is considered to be “ideal”. This algorithm builds a full isolation forest from scratch for every batch of the input stream. It follows the changing behavior perfectly, but it has high computational requirements due to the high number of isolation trees created (with sample size 40,000 the number of trees is $40,000/64 \cdot 1024 = 640,000$).
- The BWOAIF method with $\sigma_v^2 = \{0.5, 0.05, 0.005\}$ values. With the above mentioned parameters this method creates 40,000 trees in total.
- The IForestASD algorithm. Its threshold (determining when to rebuild the forest) is set such that the total number of trees built when processing the whole stream matches that of the BWOAIF method.

As shown in Figure 2, the computational demand of BWOAIF is constant over the time as the same (low) number of new trees are built after every batch. IForestASD updates the model less frequently, but when it does, it needs to rebuild all the trees, and the time and frequency of these model updates are unpredictable.

Next, the mean of the anomaly scores over the 50 repeated runs is computed. The results are depicted in Figure 3. The figure demonstrates that the IForestASD is not able to handle several kinds of concept drifts, especially

in phases 3, 4 and 5, where it does not update the forest due to its relatively low anomaly scores. As for the BWOAIF variants, too high $\sigma_v^2 = 0.5$ implies that all trees of the forest are used for the anomaly score computation, including the outdated ones. It adapts to concept drifts, but this adaptation is relatively slow due to the averaging effect resulting from the involvement of the old trees. With $\sigma_v^2 = 0.05$, the BWOAIF method is able to follow the concept drift of the stream, and with $\sigma_v^2 = 0.005$, BWOAIF reproduces the “ideal” results since only the newest trees are involved in the anomaly score computation.

Next, we investigate the variance of the anomaly scores over the 50 runs in Figure 4. IForestASD produces very high variance, that is, exhibits significant dependency on the initial random seed. BWOAIF with $\sigma_v^2 = 0.5$ gives a variance as low as the “ideal” method, since a high number of trees (all trees) are taken into account in the anomaly score computation. With $\sigma_v^2 = 0.05$, the benefit of the bilateral weighting becomes clear: when there is no or only slow concept drift, all or almost all trees are being incorporated, leading to low variance almost matching that of the ideal case. Around sudden concept drifts, however, the old, outdated trees are ignored (they have low weight), which produced a high variance for a short time period, until a sufficient number of new trees are built from the new batches. With $\sigma_v^2 = 0.005$, only the trees built from the latest batch are used, hence, the variance is rather high everywhere.

The σ_a parameter was set to a high enough number in these experiments, to make the benefit of BWOAIF as clear as possible. With lower σ_a values it is possible to prioritize newer trees over older ones if necessary.

5.2 | Comparison using benchmark data sets

In this section, we provide further comparisons based on two frequently used benchmark data sets, the *Shuttle* and the *Smtplib* data sets. The *Shuttle* data set consists of 49,097 samples and has 9 numerical attributes, while the *Smtplib* data set has 95,156 samples and 3 numerical variables.

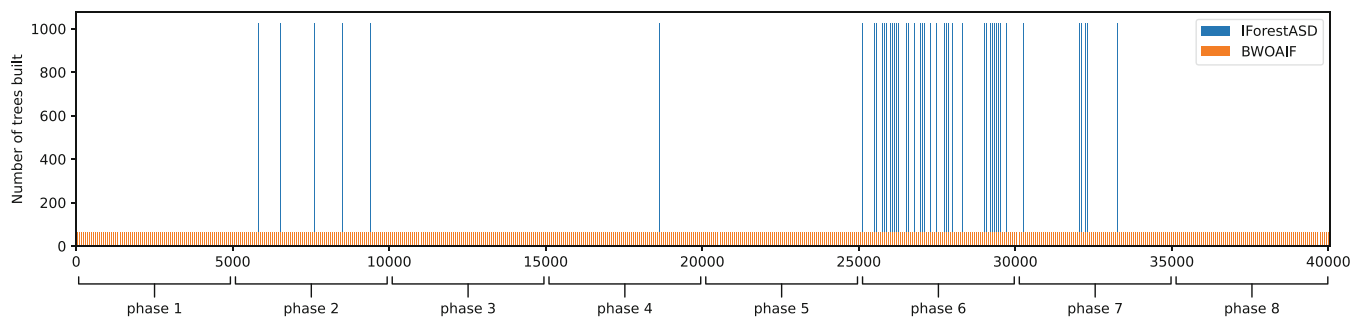


FIGURE 2 The distribution of the number of trees built by the algorithms

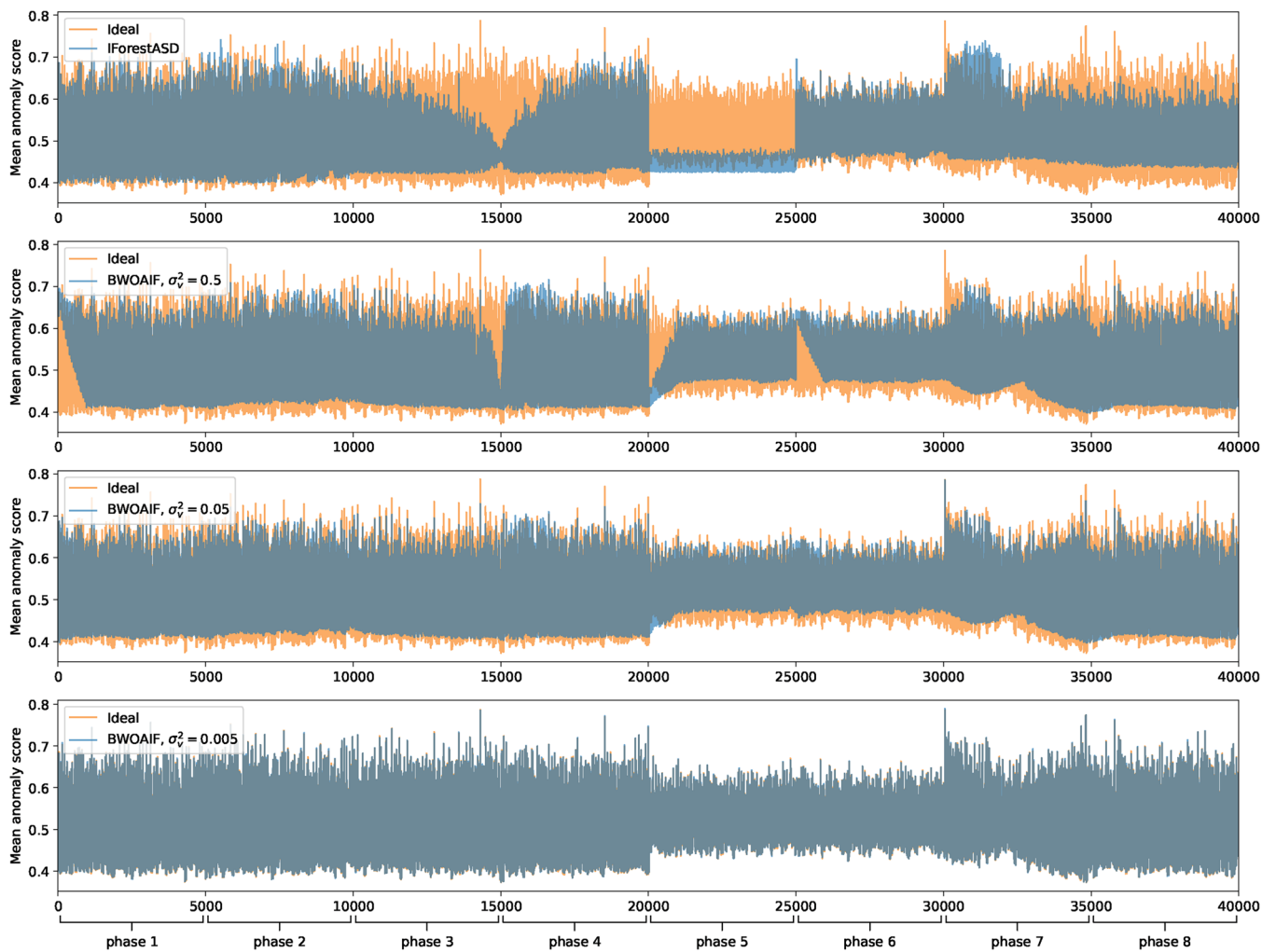


FIGURE 3 Mean of the anomaly scores based on 50 runs with different random seeds

In case of the *Shuttle* data set, we have set the batch size to $W = 64$ and the total number of trees to $T = 1024$. Table 1 compares the results of the global (non-online) isolation forest, the IForestASD, and BWOAIF with three different σ_v^2 settings. Among the online procedures, BWOAIF performs best with higher σ_v^2 values, which encourages the model to use as many trees of the ensemble as possible. The reason for this behavior could be that the *Shuttle* data set does not have abrupt concept changes.

Anomaly detection in the *Smtip* is more difficult, and similar to other research papers we arrived at lower AUC scores. For this data set, the parameters were $W = 512$ and $T = 1024$. The AUC values are summarized in Table 2. we decided to omit the F1 score, precision, and recall values since they are not meaningful with such low AUC scores. For this particular example, the BWOAIF method with $\sigma_v^2 = 0.1$ gives the highest score, providing a good balance between using all trees of the ensemble and only those similar enough to the last one.

Finally, we perform some more comparisons on further data sets such as the *NSL* data set [14] and some ODDS data sets [10] including the *Mammography*, *Ionosphere*, *Cardio*, *Satimage-2* and the *ForestCover* data sets. In this experiment the forest re-building threshold of the IForestASD is set to its default value, that is the anomaly rate. The Half-Space Tree method [13] is also involved in the comparison. We have set the batch size to $B = 64$ in all of the cases. The results are listed in Table 3.

On these data sets the BWOAIF performs significantly better than both the HS-Trees and the IForestASD, except for the *ForestCover*, where the HS-Trees achieved the best results. As in the case studies before, setting the σ_v^2 parameter as low as 0.01 has a negative impact on the results, probably because there is no abrupt concept drifts in these data. The best values were obtained with σ_v^2 values between 1 and 0.1, setting σ_v^2 higher than 1 does not give any further improvement. Note that these AUC values are not directly comparable to the results presented in past papers, since

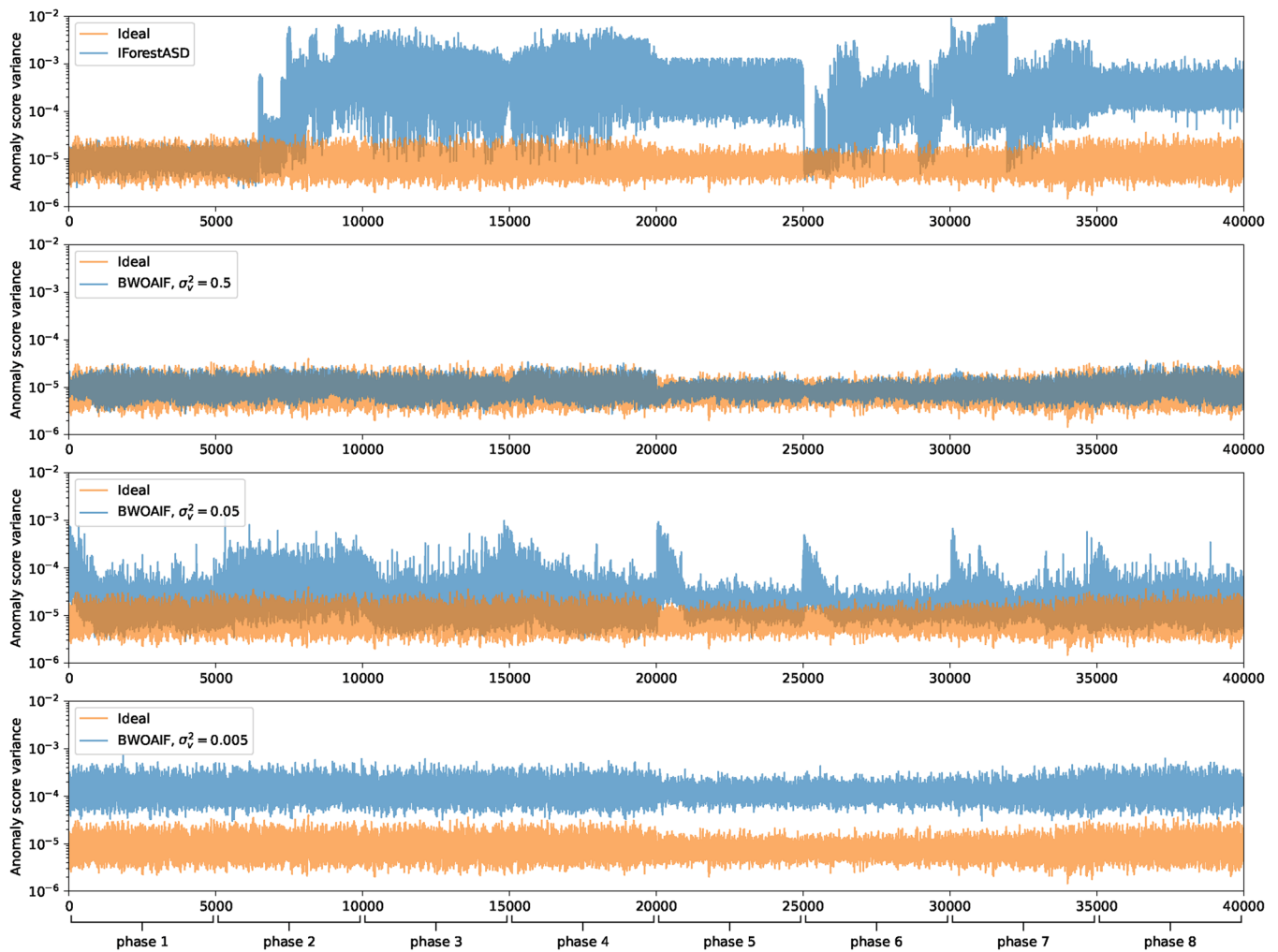


FIGURE 4 Variance of the anomaly scores based on 50 runs with different random seeds

TABLE 1 Comparison using the *Shuttle* data set

	AUC	F1 score (%)	Precision (%)	Recall (%)
Global IF method	0.996	91.08	86.45	96.23
IForestASD	0.935	93.47	93.86	93.08
BWOAIF, $\sigma_v^2 = 0.5$	0.996	96.88	98.46	95.34
BWOAIF, $\sigma_v^2 = 0.05$	0.997	95.29	95.73	94.85
BWOAIF, $\sigma_v^2 = 0.005$	0.994	87.03	87.78	86.3

TABLE 2 Comparison using the *Smtip* data set

	AUC
Global method	0.881
IForestASD	0.845
BWOAIF, $\sigma_v^2 = 1.0$	0.845
BWOAIF, $\sigma_v^2 = 0.1$	0.866
BWOAIF, $\sigma_v^2 = 0.01$	0.83

we have set a very small batch size, which turned out to be demanding for all the methods, except for the BWOAF.

6 | CONCLUSIONS

In this paper, we have presented a new Isolation Forest variant, BWOAIF, that can process the data in an online manner, updating the oldest components of the model when a batch of new data arrives. With a special

TABLE 3 AUC of the IForestASD and the BWOAIF method on various data sets (average of five executions)

Method	NSL	Mamm.	Ion.	Cardio	Sat.	Cover
IForestASD	0.649	0.625	0.758	0.487	0.671	0.437
HS-Trees	0.597	0.774	0.523	0.309	0.488	0.870
BWOAIF, $\sigma_v^2 = 1.0$	0.798	0.868	0.805	0.91	0.988	0.434
BWOAIF, $\sigma_v^2 = 0.1$	0.799	0.834	0.804	0.849	0.988	0.414
BWOAIF, $\sigma_v^2 = 0.01$	0.799	0.616	0.773	0.489	0.713	0.438

bilateral weighting scheme the model is able to prioritize those members of the ensemble which are similar to the most recent one and also prioritize those which have been trained on the most recent data. As a consequence of these two independent weighting schemes BWOAIF can cope with smooth as well as abrupt concept changes. It is important to emphasize that there are no hard decisions in this procedure, e.g., no explicit concept drift detection, which results in relatively smooth and coherent anomaly scores, and implies balanced, predictable computational requirements. Numerical results show the stability of BWOAIF in terms of anomaly score distribution and of its computational requirements, and how it copes with different types of concept drifts. We hypothesize that the proposed bilateral weighting has benefits in other online ensemble methods as well.

FUNDING INFORMATION

This work was supported by the European Union project RRF-2.3.1-21-2022-00004 within the framework of the Artificial Intelligence National Laboratory. The research was partially supported by the OTKA 138208 project.

CONFLICT OF INTEREST

The authors have no conflicts of interest to declare.

DATA AVAILABILITY STATEMENT

The algorithm to generate the synthetic data set is clearly detailed in the paper. The “Shuttle” data set is available to download from the UCI Machine Learning Repository [https://archive.ics.uci.edu/ml/datasets/Statlog+\(Shuttle\)](https://archive.ics.uci.edu/ml/datasets/Statlog+(Shuttle)), and the Smtip data set can be downloaded from <https://riverml.xyz/dev/api/datasets/SMTP/>.

REFERENCES

- R. Chalapathy and S. Chawla, Deep learning for anomaly detection: a survey. arXiv preprint arXiv:1901.03407, 2019.
- V. Chandola, A. Banerjee, and V. Kumar, *Anomaly detection: A survey*, ACM Comput. Surv. 41 (2009), no. 3, 1–58.
- V. Chatzigiannakis, S. Papavassiliou, M. Grammatikou, and B. Maglaris, Hierarchical anomaly detection in distributed large-scale sensor networks. In 11th IEEE Symposium on Computers and Communications (ISCC'06), pages 761–767. IEEE, 2006.
- Z. Ding and M. Fei, *An anomaly detection approach based on isolation forest algorithm for streaming data using sliding window*, IFAC Proceed. Vol. 46 (2013), no. 20, 12–17.
- R. Gwadera, M. J. Atallah, and W. Szpankowski, *Reliable detection of episodes in event sequences*, Knowled. Inform. Syst. 7 (2005), no. 4, 415–437.
- R. A. A. Habeeb, F. Nasaruddin, A. Gani, I. A. T. Hashem, E. Ahmed, and M. Imran, *Real-time big data processing for anomaly detection: A survey*, Int. J. Inform. Manag. 45 (2019), 289–307.
- J. Z. Kolter and M. A. Maloof, *Dynamic weighted majority: An ensemble method for drifting concepts*, J. Mach. Learn. Res. 8 (2007), 2755–2790.
- J. Lin, E. Keogh, A. Fu, and H. Van Herle, Approximations to magic: Finding unusual medical time series. In 18th IEEE Symposium on Computer-Based Medical Systems (CBMS'05), pages 329–334. IEEE, 2005.
- F. T. Liu, K. M. Ting, and Z.-H. Zhou, Isolation forest. In 2008 Eighth IEEE International Conference on Data Mining, pages 413–422. IEEE, 2008.
- S. Rayana. Odds library, 2016.
- J. C. Schlimmer and R. H. Granger, Beyond incremental processing: Tracking concept drift. In AAAI, 502–507, 1986.
- C. Spence, L. Parra, and P. Sajda, Detection, synthesis and compression in mammographic image analysis with a hierarchical image probability model. In Proceedings IEEE workshop on mathematical methods in biomedical image analysis (MMBIA 2001), pages 3–10. IEEE, 2001.
- S. C. Tan, K. M. Ting, and T. F. Liu, Fast anomaly detection for streaming data. In Twenty-second international joint conference on artificial intelligence, 2011.
- M. Tavallae, E. Bagheri, W. Lu, and A. A. Ghorbani, A detailed analysis of the kdd cup 99 data set. In 2009 IEEE symposium on computational intelligence for security and defense applications, pages 1–6. IEEE, 2009.
- M. U. Togbe, Y. Chabchoub, A. Boly, M. Barry, R. Chiky, and M. Bahri, *Anomalies detection using isolation in concept-drifting data streams*, Computers 10 (2021), no. 1, 13.
- C. Tomasi and R. Manduchi, Bilateral filtering for gray and color images. In Sixth international conference on computer vision (IEEE Cat. No. 98CH36271), pages 839–846. IEEE, 1998.

How to cite this article: G. Hannák, G. Horváth, A. Kádár, and M. D. Szalai, *Bilateral-Weighted Online Adaptive Isolation Forest for anomaly detection in streaming data*, Stat. Anal. Data Min.: ASA Data Sci. J. **16** (2023), 215–223. <https://doi.org/10.1002/sam.11612>